Analiza algorytmów dla Problemu Komiwojażera (ATSP)

Bartłomiej Czyż

16 marca 2026

Spis treści

Wstęp **2**

3 Algorytm losowy (RAND) **2**
3.1 Liczba iteracji a jakość rozwiązania 2
3.2 Złożoność obliczeniowa i maksymalny rozmiar instancji 2
3.3 Minimalizacja powtarzalności permutacji 3

4 Algorytmy zachłanne (NN i RNN) **3**
4.1 Analiza czasu wykonania i błędu względnego 3
4.2 Złożoność obliczeniowa i limit 15 minut 5
4.3 Wpływ cech grafu na złożoność zbadanych algorytmów 5
4.4 Ograniczenia Górne (UB) i Dolne (LB) 6

5 Przegląd zupełny (Brute Force) **6**
5.1 Analiza czasu wykonania i złożoność obliczeniowa 6
5.2 Możliwości modyfikacji algorytmu 7
5.3 Porównanie z algorytmami NN, RNN i RAND 7

Wstęp

Celem projektu jest analiza i porównanie efektywności algorytmów rozwiązujących Problem Komiwojażera (ATSP). W badaniach zaimplementowano i przetestowano algorytmy: przeglądu zupełnego (Brute Force), zachłanne (NN, RNN) oraz algorytm losowy (RAND). Analizie poddano jakość wyznaczanych tras (błąd względny) oraz złożoność czasową w zależności od rozmiaru instancji N.

3 Algorytm losowy (RAND)

3.1 Liczba iteracji a jakość rozwiązania

Celem badania było określenie minimalnej liczby iteracji (wygenerowanych losowych ścieżek), która pozwoli algorytmowi RAND na znalezienie rozwiązania z błędem względnym poniżej 10% w stosunku do znanego optimum. Zestawienie wyników uśrednionych z 10 prób dla badanych instancji problemu ATSP przedstawia poniższa tabela.

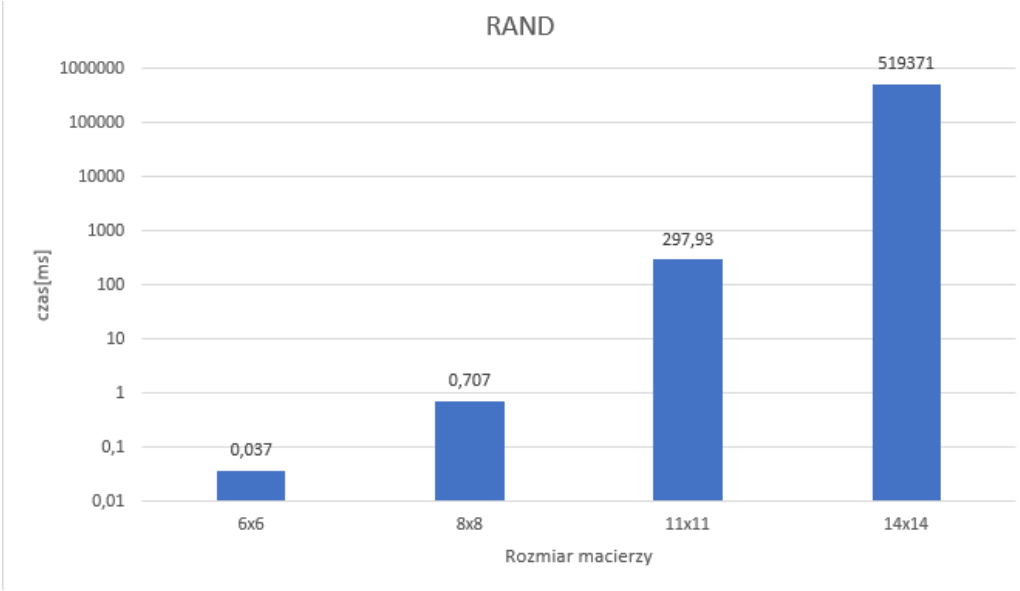
Tabela 1: Zależność liczby iteracji, średniego błędu i czasu od rozmiaru instancji N

Rozmiar instancji (N)	**Liczba iteracji (i)**	**Średni błąd [%]**	**Średni czas**
6x6	190	0,80	0,037 ms
8x8	5 000	3,02	0,707 ms
11x11	1 500 000	2,35	297,93 ms
14x14	2 100 000 000	5,83	519,37 s

3.2 Złożoność obliczeniowa i maksymalny rozmiar instancji

Złożoność obliczeniowa pojedynczej iteracji wynosi $O(N)$, ponieważ wygenerowanie i zsumowanie kosztów trasy wymaga przejścia przez wszystkie wierzchołki. Dla k z góry ustalonych iteracji, złożoność to $O(k \cdot N)$.

Jednakże, aby utrzymać sensowny wynik (błąd poniżej 10%), liczba iteracji k musi rosnąć proporcjonalnie do wielkości przestrzeni poszukiwań. Przestrzeń ta rośnie w tempie silniowym, $O(N!)$. Zatem **rzeczywista złożoność algorytmu RAND** dla zachowania zadanego progu dokładności jest silniowa: $O(N \cdot N!)$.



Rysunek 1: Zależność średniego czasu uzyskania rozwiązania od rozmiaru instancji (skala logarytmiczna)

Zależność czasu uzyskania rozwiązania od rozmiaru instancji rośnie drastycznie (co obrazuje Rysunek 1). Dla zachowania błędu poniżej 10%, rozmiar **14x14** jest maksymalną instancją, dla której udaje się znaleźć rozwiązanie w rozsądnym czasie 15 minut (osiągnięty średni czas to około 8,6 minuty dla 2,1 miliarda iteracji). Dla instancji 15x15 liczba potrzebnych iteracji znacznie przekroczyłaby ten limit.

3.3 Minimalizacja powtarzalności permutacji

Aby generować kolejne losowe permutacje w sposób minimalizujący szansę na ich powtórzenie (przy zachowaniu czysto losowego charakteru i bez używania algorytmów systematycznie generujących kolejne permutacje leksykograficzne), należy zastosować **algorytm tasowania Fishera-Yatesa (tasowanie Knutha)**. Metoda ta polega na iterowaniu po tablicy miast od końca i zamianie bieżącego elementu z losowo wybranym elementem z puli nieprzetasowanych jeszcze elementów (włączając w to sam bieżący element). Gwarantuje ona, że każda z $N!$ permutacji ma dokładnie takie samo prawdopodobieństwo wystąpienia, eliminuje błędy obciążenia (ang. bias) charakterystyczne dla naiwnych algorytmów losujących i działa w optymalnym czasie liniowym $O(N)$.

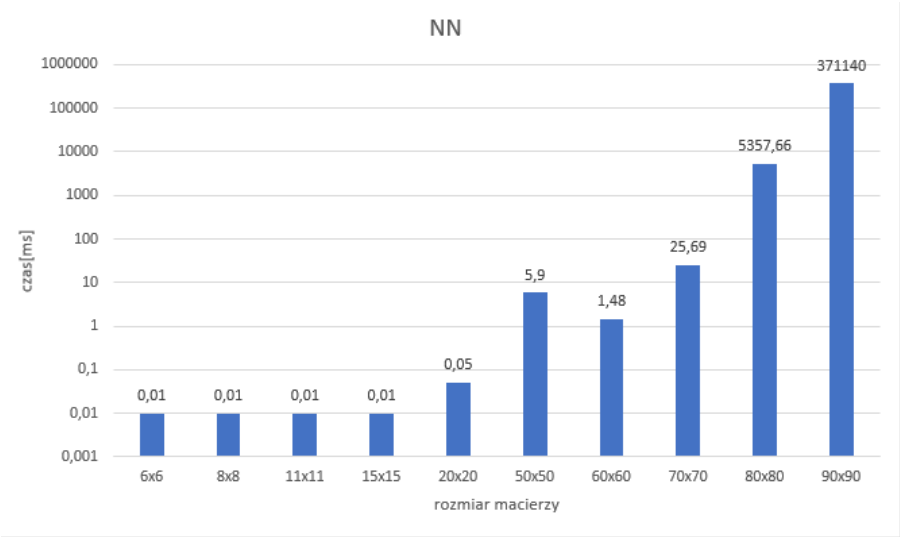
4 Algorytmy zachłanne (NN i RNN)

4.1 Analiza czasu wykonania i błędu względnego

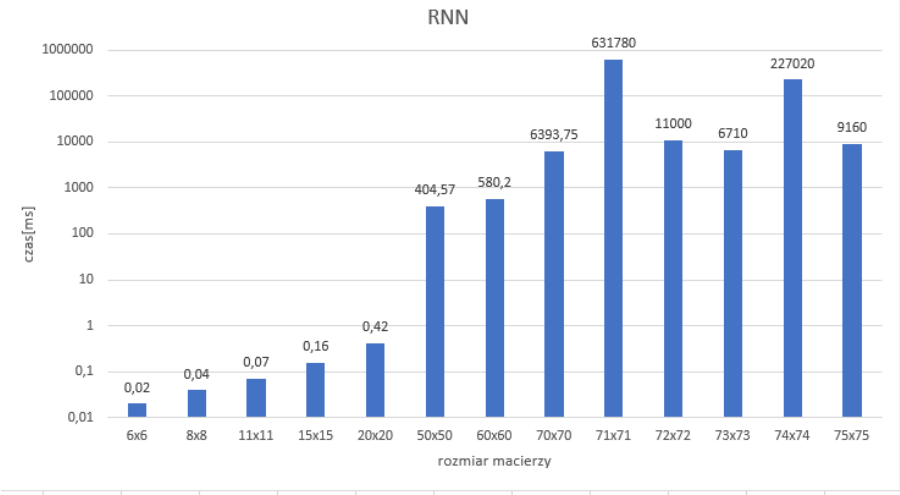
Algorytm Najbliższego Sąsiada (NN - *Nearest Neighbor*) oraz powtarzalny NN (RNN - *Repetitive Nearest Neighbor*) zostały zaimplementowane z uwzględnieniem warunku sprawdzania każdej ścieżki w przypadku napotkania krawędzi o identycznych, minimalnych wagach (rozgałęzienie poszukiwań).

Tabela 2: Porównanie czasu wykonania i błędu dla algorytmów NN i RNN

Rozmiar	**Czas NN**	**Błąd NN [%]**	**Czas RNN**	**Błąd RNN [%]**
6x6	<0,01 ms	8,00	0,02 ms	0,00
8x8	0,01 ms	11,76	0,04 ms	6,62
11x11	0,01 ms	62,42	0,07 ms	7,35
15x15	0,01 ms	46,95	0,16 ms	28,05
20x20	0,05 ms	66,84	0,42 ms	35,75
50x50	5,90 ms	-	404,57 ms	-
60x60	1,48 ms	-	580,20 ms	-
70x70	25,69 ms	-	6393,75 ms	-
71x71	nie testowany	-	631,78 s	-
72x72	nie testowany	-	11,00 s	-
73x73	nie testowany	-	6,71 s	-
74x74	nie testowany	-	227,02 s	-
75x75	nie testowany	-	9,16 s	-
76x76	nie testowany	-	>15 min	-
80x80	5357,66 ms	-	nie testowany	-
90x90	371,14 s	-	nie testowany	-
91x91	1605,50 s	-	nie testowany	-



Rysunek 2: Porównanie czasów algorytmu NN



Rysunek 3: Porównanie czasów algorytmu RNN

RNN zazwyczaj oferuje rozwiązania o zauważalnie niższym błędzie względnym niż NN. Wynika to z faktu, że algorytm NN jest silnie uzależniony od losowo wybranego wierzchołka startowego (tzw. zachłanność krótkowzroczna). RNN, startując z każdego wierzchołka i wybierając najlepszą z tras, niweluje problem „pechowego startu".

4.2 Złożoność obliczeniowa i limit 15 minut

Standardowa złożoność algorytmu NN to $O(N^2)$, a RNN to $O(N^3)$. Jednakże wymóg rozgałęziania ścieżek przy krawędziach o tych samych wagach zmienia charakter algorytmu. W pesymistycznym przypadku (wiele identycznych wag) algorytmy degradują się do złożoności wykładniczej $O(b^N)$, gdzie b to średni współczynnik rozgałęzienia.

Skokowe wzrosty czasów w tabeli (np. skok czasu RNN z 11 s dla 72x72 do ponad 227 s dla 74x74) są bezpośrednim wynikiem tej degradacji – czas nie zależy już tylko od N, ale od struktury wag konkretnej instancji. Na podstawie wyników, **maksymalny rozmiar instancji** możliwy do rozwiązania w 15 minut (900 000 ms) wynosi około $95x95$ dla NN oraz około $70x70 - 75x75$ dla RNN (w zależności od liczby powtarzających się wag). Limit czasowy 15 minut jest konieczny. W algorytmie losowym zapobiega on nieskończonemu poszukiwaniu przy złożoności $O(N \cdot N!)$, natomiast w NN/RNN stanowi zabezpieczenie przed "eksplozją kombinatoryczną"wywołaną kaskadowym rozgałęzianiem w instancjach o gęstym występowaniu identycznych wag.

4.3 Wpływ cech grafu na złożoność zbadanych algorytmów

- **Gęstość grafu:** W grafach pełnych (badane ATSP) algorytm w każdym kroku musi sprawdzić $N - 1$ krawędzi. Spadek gęstości (brak niektórych połączeń) przyspieszyłby działanie zachłanne (mniej opcji do weryfikacji), ale zwiększyłby ryzyko utknięcia w "ślepym zaułku"i braku znalezienia cyklu Hamiltona.

- **Istnienie wielu ścieżek optymalnych:** Dla standardowego NN/RNN nie ma to wpływu na czas. Jednak przy wymogu rozgałęziania, wiele równoważnych ścieżek zmusza algorytm do ich jednoczesnego eksplorowania, co drastycznie podnosi czas wykonania.

- **Zakres wag krawędzi:** Jest to czynnik krytyczny dla zaimplementowanej wersji. Wąski zakres (np. 1-10) generuje ogromną liczbę identycznych wag krawędzi, powodując gigantyczne rozgałęzienia i spadek wydajności. Szeroki zakres (np. 1-10000) sprawia, że

wagi są unikalne, algorytm nie musi się rozgałęziać i działa z optymalną, wielomianową złożonością.

4.4 Ograniczenia Górne (UB) i Dolne (LB)

Górne Ograniczenie (UB) to koszt dowolnej, poprawnej trasy komiwojażera (rozwiązanie dopuszczalne). Algorytmy zachłanne (NN, RNN) są niezwykle efektywnymi metodami do wyznaczania UB. Mimo lokalnych wahań czasu przez narzucone rozgałęzienia, znajdują one bardzo dobre (niskie) UB w czasie nieporównywalnie krótszym niż metoda Brute Force.

Dolne Ograniczenie (LB) to teoretyczna wartość, poniżej której nie da się zejść, często niebędąca poprawną trasą TSP (np. wyznaczana poprzez algorytm węgierski dla problemu przydziału lub relaksację opartą na 1-drzewie). Badane algorytmy zachłanne i losowe nie wyznaczają wartości LB.

5 Przegląd zupełny (Brute Force)

5.1 Analiza czasu wykonania i złożoność obliczeniowa

Badaniu poddano algorytm przeglądu zupełnego oraz jego zoptymalizowaną wersję wykorzystującą technikę odcinania gałęzi (*Branch & Bound*). Testy przeprowadzono dla grafów asymetrycznych i symetrycznych.

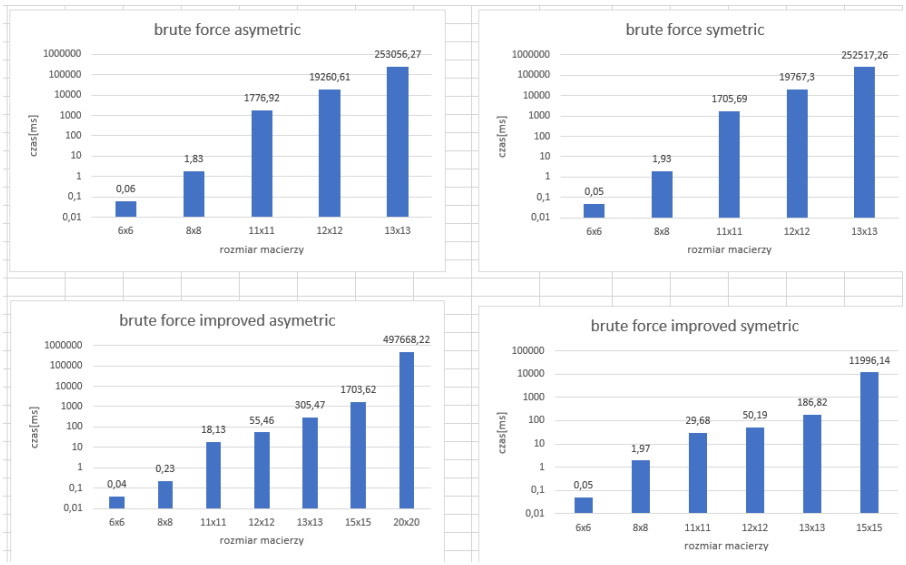
Tabela 3: Czas wykonania algorytmów przeglądu zupełnego [ms]

Rozmiar	**Asym. (Czysty)**	**Sym. (Czysty)**	**Asym. (B&B)**	**Sym. (B&B)**
6x6	0,06	0,05	0,04	0,05
8x8	1,83	1,93	0,23	1,97
11x11	1776,92	1705,69	18,13	29,68
12x12	19260,61	19767,30	55,46	50,19
13x13	253056,30	252517,30	305,47	186,82
15x15	-	-	1703,62	11996,14
20x20	-	-	497668,20	>30 min

Na podstawie wyników dla „czystego” algorytmu można zauważyć, że czasy dla instancji asymetrycznych i symetrycznych są niemal identyczne. Wynika to z faktu, że algorytm ten jest „ślepy” na strukturę grafu i zawsze musi wygenerować wszystkie permutacje. Wyznaczona na tej podstawie **złożoność obliczeniowa wynosi** $O(N!)$.

Czas rozwiązania rośnie w tempie silniowym. Przejście z rozmiaru 12 na 13 wydłużyło czas trzynastokrotnie (z ∼19 sekund do ∼4,2 minuty). Szacując na tej podstawie czas dla instancji 14x14, wyniósłby on około 60 minut. Zatem **maksymalnym rozmiarem instancji**, dla której czysty algorytm znajduje rozwiązanie w rozsądnym czasie 30 minut, jest **13x13**. Z kolei dla algorytmu zoptymalizowanego (*Branch & Bound*) maksymalny rozmiar instancji zależy drastycznie od struktury grafu. Dla problemu asymetrycznego algorytm rozwiązał instancję 20x20 w czasie około 8,3 minuty. Jednakże dla problemu symetrycznego jego wydajność gwałtownie spada. Dodatkowe testy empiryczne wykazały, że rozwiązanie wszystkich badanych instancji w przedziale od 16x16 do 20x20 przekroczyło zadany limit 30 minut. Zatem maksymalnym rozmiarem instancji symetrycznej, dla której zoptymalizowany algorytm B&B bezbłędnie znajduje rozwiązanie w akceptowalnym czasie, jest zaledwie 15x15 (czas ok. 12 sekund).

Ten drastyczny spadek wydajności B&B dla macierzy symetrycznych wynika ze zjawiska braku łamania symetrii (tzw. lustrzanych odbić). W grafie asymetrycznym każda trasa jest zazwyczaj unikalna kosztowo. W grafie symetrycznym każda wygenerowana ścieżka ma swojego bliźniaka – czytana od tyłu ma dokładnie ten sam koszt. Ponieważ zaimplementowana podstawowa wersja B&B nie posiada mechanizmu rozpoznawania i odrzucania rewersów, algorytm traci ogromną ilość czasu na głęboką analizę symetrycznych duplikatów tras, które powinien od razu odrzucić. Znacznie osłabia to siłę przycinania drzewa poszukiwań.



Rysunek 4: Porównanie czasów algorytmów brute force

5.2 Możliwości modyfikacji algorytmu

Algorytm przeglądu zupełnego można znacząco zmodyfikować, by poprawić jego efektywność czasową: **Zastosowanie techniki Branch & Bound (odcinanie permutacji, gdy częściowy koszt trasy przekroczy dotychczas najlepszy znaleziony)** drastycznie redukuje przestrzeń poszukiwań. Jak widać w tabeli, pozwala to na rozwiązanie problemu 20x20 w około 8,3 minuty (dla macierzy asymetrycznej).

5.3 Porównanie z algorytmami NN, RNN i RAND

Algorytm przeglądu zupełnego jako jedyny z badanych gwarantuje odnalezienie rozwiązania optymalnego (błąd względny 0%). Odbywa się to jednak kosztem drastycznie wyższej złożoności czasowej $O(N!)$ w stosunku do wielomianowych algorytmów zachłannych ($O(N^2)$ dla NN, $O(N^3)$ dla RNN).

Algorytmy zachłanne i losowe skalują się o wiele lepiej – pozwalają na analizę macierzy o rozmiarach rzędu 70x70 czy 90x90 w czasie, w którym przegląd zupełny nie jest w stanie wyjść poza rozmiar 14x14. Sprawia to, że Brute Force ma zastosowanie wyłącznie teoretyczne lub dla trywialnie małych instancji, podczas gdy w zastosowaniach inżynierskich preferowane są heurystyki (jak badane RNN), które za cenę niewielkiego błędu dostarczają wynik w akceptowalnym czasie.